


湖北汽车工业学院
HUBEI UNIVERSITY OF AUTOMOTIVE TECHNOLOGY

# 智能网联汽车学院

## 移动终端软件开发

# 实验指导书

制定人：张　毅

审核人：张吴波

日　期：2026年8月

# 目 录

目 录.....1
实验四、图形图像和多媒体技术.....2
**一、实验内容**.....2
**二、思考题**.....9

# 实验四、图形图像和多媒体技术

## 一、实验内容

### 程序 1 Paint 与 Canvas 绘图

（1）写出自定义 View 的 onDraw 方法，以及设置 Paint 属性、使用 Canvas 绘制二维图形的关键代码。

说明：程序位于 com.example.graphicsmedia 包，PaintCanvasActivity 加载布局 activity_paint_canvas.xml，其中放置自定义 View（ShapeView），绘制工作全部在 onDraw() 中完成。

```
// ShapeView.java —— 自定义 View，重写 onDraw() 用 Paint + Canvas 绘制二维图形
public class ShapeView extends View {
    private final Paint paint = new Paint(Paint.ANTI_ALIAS_FLAG);      // 主画笔：抗锯齿
    private final Paint labelPaint = new Paint(Paint.ANTI_ALIAS_FLAG);  // 注释文字画笔

    public ShapeView(Context context, AttributeSet attrs) { super(context, attrs); }

    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        canvas.drawColor(Color.WHITE);             // 画布背景
        float scale = getWidth() / 1080f;          // 以 1080px 为设计基准等比缩放
        canvas.save();
        canvas.scale(scale, scale);

        paint.setAntiAlias(true);                 // 抗锯齿
        labelPaint.setTextSize(28);
        labelPaint.setColor(Color.DKGRAY);

        // ① 实心矩形：FILL + setColor
        paint.setStyle(Paint.Style.FILL);
        paint.setColor(Color.rgb(0xFF, 0x57, 0x22)); // 橙色
        canvas.drawRect(60, 150, 480, 360, paint);

        // ② 描边矩形：STROKE + setStrokeWidth
        paint.setStyle(Paint.Style.STROKE);
        paint.setStrokeWidth(14);                   // 笔触宽度 14px
        paint.setColor(Color.rgb(0x21, 0x96, 0xF3));
        canvas.drawRect(600, 150, 1020, 360, paint);

        // ③ 实心圆与圆环
        paint.setStyle(Paint.Style.FILL);
        paint.setColor(Color.rgb(0x4C, 0xAF, 0x50));
        canvas.drawCircle(260, 560, 120, paint);     // 实心圆
        paint.setStyle(Paint.Style.STROKE);
        paint.setStrokeWidth(16);
        paint.setColor(Color.rgb(0x9C, 0x27, 0xB0));
        canvas.drawCircle(760, 560, 120, paint);     // 圆环

        // ④ 椭圆与扇形（drawArc：起始角 30°，扫过 270°，useCenter=true）
        paint.setStyle(Paint.Style.FILL);
        paint.setColor(Color.rgb(0xE9, 0x1E, 0x63));
        oval.set(60, 780, 500, 980);
        canvas.drawOval(oval, paint);
```

```
            paint.setColor(Color.rgb(0x00, 0xAC, 0xC1));
            oval.set(600, 780, 1020, 980);
            canvas.drawArc(oval, 30, 270, true, paint);

            // ⑤ 直线、点、圆角矩形
            paint.setStyle(Paint.Style.STROKE);
            paint.setStrokeWidth(10);
            paint.setStrokeCap(Paint.Cap.ROUND);        // 圆头线帽
            paint.setColor(Color.rgb(0x3F, 0x51, 0xB5));
            canvas.drawLine(60, 1180, 480, 1080, paint);
            paint.setStrokeWidth(46);                   // 笔触宽度决定点的大小
            paint.setColor(Color.rgb(0xF4, 0x43, 0x36));
            canvas.drawPoint(700, 1120, paint);
            canvas.drawPoint(860, 1160, paint);
            paint.setStyle(Paint.Style.FILL);
            paint.setColor(Color.rgb(0x8B, 0xC3, 0x4A));
            canvas.drawRoundRect(600, 1200, 1020, 1280, 40, 40, paint);

            // ⑥ 路径 Path：三角形、五角星
            paint.setStyle(Paint.Style.FILL);
            paint.setColor(Color.rgb(0xFF, 0xC1, 0x07));
            path.reset();
            path.moveTo(230, 1360);
            path.lineTo(60, 1600);
            path.lineTo(400, 1600);
            path.close();
            canvas.drawPath(path, paint);               // 沿用同一个 Path 画五角星（略）

            // ⑦ 文本：setTextSize + setFakeBoldText
            paint.setColor(Color.rgb(0x21, 0x21, 0x21));
            paint.setTextSize(56);
            paint.setFakeBoldText(true);
            canvas.drawText("Android 绘图：Paint + Canvas", 60, 1830, paint);

            canvas.restore();
        }
    }
```

（2）粘贴绘图结果截图，标注图形类型及主要画笔属性。

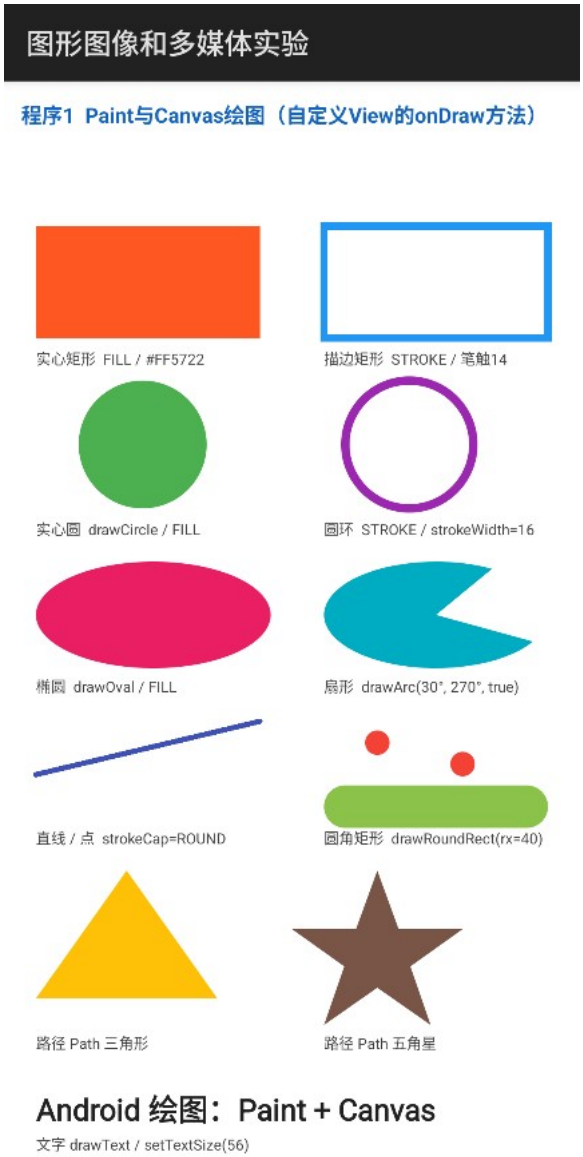



图 1-1 程序 1 运行结果（Android 15 模拟器，1080×2400）

图形类型与画笔属性已在图中标注：实心/描边矩形（FILL/STROKE、strokeWidth）、实心圆/圆环、椭圆/扇形（drawArc）、直线/点（strokeCap）、圆角矩形、路径（drawPath）、文本（setTextSize/setFakeBoldText）

## 程序 2 Matrix 图像变换

（1）写出利用 Matrix 实现平移、旋转、缩放、扭曲变换，并通过 Canvas 显示图像的关键代码。

说明：MatrixActivity 使用自定义 View（MatrixView），在同一画布上分 5 个面板，用 Matrix + Canvas.drawBitmap(bitmap, matrix, paint) 显示原图与四种变换结果。

```
// MatrixView.java —— 用 Matrix 对同一张图片做平移、旋转、缩放、扭曲变换
```

```
public class MatrixView extends View {
    private Bitmap bitmap;                    // res/drawable-nodpi/rabbit.png
    private final Matrix matrix = new Matrix();
    private final Paint bitmapPaint = new Paint(Paint.ANTI_ALIAS_FLAG);

    private void init() {
        bitmap = BitmapFactory.decodeResource(getResources(), R.drawable.rabbit);
        bitmapPaint.setFilterBitmap(true);      // 变换后开启图像过滤，避免锯齿
    }

    @Override
    protected void onDraw(Canvas canvas) {
        canvas.drawColor(Color.WHITE);
        for (int i = 0; i < 5; i++) {           // 5 个面板：原图 / 平移 / 旋转 / 缩放 / 扭曲
            float top = i * 360f;
            canvas.save();
            canvas.clipRect(0, top, 1080, top + 360);
            canvas.translate(0, top);
            canvas.drawText(titles[i], 30, 50, titlePaint);
            drawTransformedBitmap(canvas, i, 540f, 220f);
            canvas.restore();
        }
    }

    private void drawTransformedBitmap(Canvas canvas, int index, float cx, float cy) {
        float bw = bitmap.getWidth();
        float bh = bitmap.getHeight();
        float baseScale = Math.min(560f / bw, 230f / bh);

        matrix.reset();
        matrix.postScale(baseScale, baseScale);                    // 基础缩放
        matrix.postTranslate(cx - bw * baseScale / 2f,             // 平移到面板中心
                             cy - bh * baseScale / 2f);
        switch (index) {
            case 1:  // 平移：向右 160px、向下 20px
                matrix.postTranslate(160, 20);
                break;
            case 2:  // 旋转：绕面板中心点旋转 30°
                matrix.postRotate(30, cx, cy);
                break;
            case 3:  // 缩放：以面板中心为轴放大 1.6 倍
                matrix.postScale(1.6f, 1.6f, cx, cy);
                break;
            case 4:  // 扭曲（斜切）：x 方向 0.35，y 方向 0.05
                matrix.postSkew(0.35f, 0.05f, cx, cy);
                break;
            default: // 原图，不叠加变换
                break;
        }
        canvas.drawBitmap(bitmap, matrix, bitmapPaint);
    }
}
```

（2）粘贴原图与各类变换后的运行截图，标注变换方式和参数。

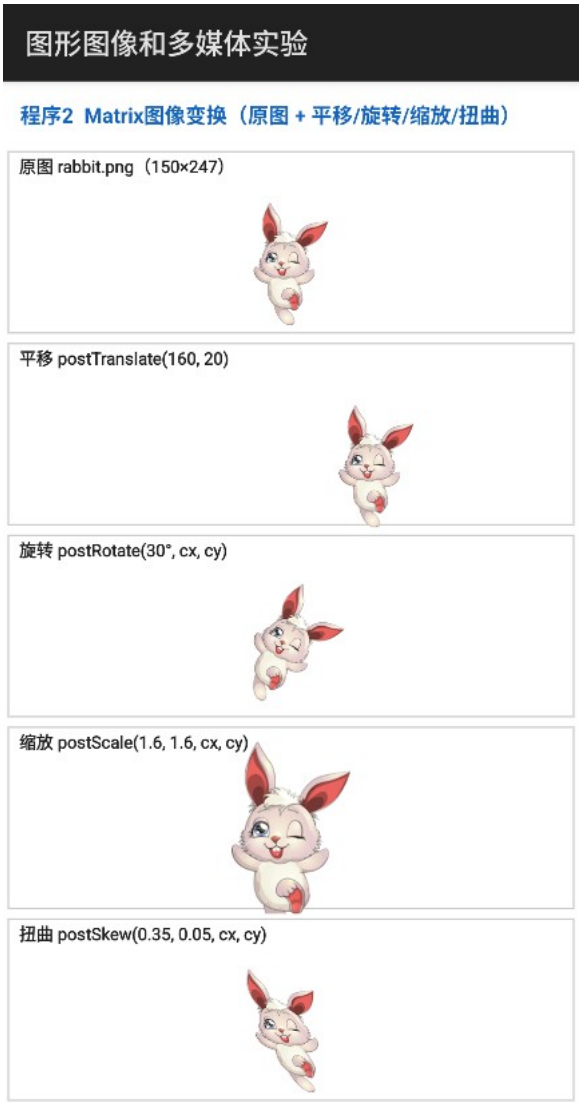



图 2-1 程序 2 运行结果：原图与平移/旋转/缩放/扭曲变换

原图 rabbit.png（150×247）；平移 postTranslate(160,20)；旋转 postRotate(30°，cx，cy)；缩放 postScale(1.6，1.6，cx，cy)；扭曲 postSkew(0.35，0.05，cx，cy)

## 程序 3 帧动画

（1）写出 animation-list 动画资源配置，以及加载、启动帧动画的关键代码，标明每帧时长和是否循环。

说明：帧动画由 res/anim/album_list.xml（animation-list）定义 10 帧 p01~p10，每帧 1000ms，oneshot="false" 表示循环播放；Android 代码中取 AnimationDrawable 后 start()/stop()。

```
<!-- res/anim/album_list.xml：帧动画资源配置（10 帧，每帧 1000ms，循环播放） -->
<?xml version="1.0" encoding="utf-8"?>
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="false">
```

```
        <item android:drawable="@drawable/p01" android:duration="1000" />
        <item android:drawable="@drawable/p02" android:duration="1000" />
        <item android:drawable="@drawable/p03" android:duration="1000" />
        <!--  …… p04 ~ p09 依次同上，共 10 帧 …… -->
        <item android:drawable="@drawable/p10" android:duration="1000" />
    </animation-list>

    <!-- res/layout/activity_frame.xml（关键部分） -->
    <ImageView
        android:id="@+id/ivFrame"
        android:layout_width="320dp"
        android:layout_height="243dp"
        android:background="#EEEEEE" />      <!-- 动画资源也可在布局中 android:background="@anim/album_list" -->
    <Button android:id="@+id/btnFrameStart" android:text="开始播放" ... />
    <Button android:id="@+id/btnFrameStop"  android:text="停止播放" ... />

    // FrameAnimActivity.java —— 加载并启动帧动画
    public class FrameAnimActivity extends Activity implements View.OnClickListener {
        private ImageView imageView;
        private AnimationDrawable frameAnim;
        private TextView tvState;

        @Override
        protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_frame);
            imageView = (ImageView) findViewById(R.id.ivFrame);
            tvState = (TextView) findViewById(R.id.tvFrameState);

            imageView.setBackgroundResource(R.anim.album_list);      // ① 设置 animation-list 为背景
            frameAnim = (AnimationDrawable) imageView.getBackground(); // ② 取出 AnimationDrawable
            tvState.setText("已加载 animation-list：共 " + frameAnim.getNumberOfFrames()
                    + " 帧，每帧 1000ms，循环播放");

            findViewById(R.id.btnFrameStart).setOnClickListener(this);
            findViewById(R.id.btnFrameStop).setOnClickListener(this);
        }

        @Override
        public void onClick(View v) {
            if (v.getId() == R.id.btnFrameStart) {
                if (!frameAnim.isRunning()) {
                    frameAnim.start();          // ③ 启动帧动画（须在窗口可见后调用）
                    tvState.setText("正在播放：p01→p10 逐帧循环，每帧 1000ms");
                }
            } else if (v.getId() == R.id.btnFrameStop) {
                if (frameAnim.isRunning()) {
                    frameAnim.stop();           // ④ 停止帧动画
                    tvState.setText("帧动画已停止（当前停留在停止时的那一帧）");
                }
            }
        }
    }
```

（2）粘贴相册播放过程中不同图片的运行截图，简述播放顺序与效果。




图 3-1 帧动画播放中的第 1 张相片
（p01 帧，ImageView 背景为 animation-list）




图 3-2 约 3.2s 后的另一帧相片
（播放顺序 p01→p02→…→p10 循环，每帧 1000ms）

## 程序 4 补间动画

（1）写出透明度、缩放、平移、旋转动画的资源配置，以及加载、启动动画的关键代码。

说明：四个基本补间动画（透明度、缩放、平移、旋转）与组合动画均定义在 res/anim 目录，由 AnimationUtils.loadAnimation() 加载、View.startAnimation() 启动。

```
<!-- ① res/anim/alpha.xml：透明度动画 1.0 → 0.2，持续 2000ms -->
<alpha xmlns:android="http://schemas.android.com/apk/res/android"
    android:duration="2000" android:fromAlpha="1.0" android:toAlpha="0.2" />


<!-- ② res/anim/scale.xml：缩放动画 1.0 → 2.0 倍，以图片中心为轴 -->
<scale xmlns:android="http://schemas.android.com/apk/res/android"
    android:duration="2000" android:fromXScale="1.0" android:toXScale="2.0"
    android:fromYScale="1.0" android:toYScale="2.0"
    android:pivotX="50%" android:pivotY="50%" />


<!-- ③ res/anim/translate.xml：平移动画 (0,0) → (300,300) -->
<translate xmlns:android="http://schemas.android.com/apk/res/android"
    android:duration="2000" android:fromXDelta="0" android:toXDelta="300"
    android:fromYDelta="0" android:toYDelta="300" />


<!-- ④ res/anim/rotate.xml：旋转动画 0° → 360°，以图片中心为轴 -->
<rotate xmlns:android="http://schemas.android.com/apk/res/android"
    android:duration="2000" android:fromDegrees="0" android:toDegrees="360"
    android:pivotX="50%" android:pivotY="50%" />
```

```
<!-- ⑤ res/anim/anim_set.xml：组合动画（按指导书参数，四种变换同时播放） -->
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <alpha android:duration="5000" android:fromAlpha="0.0" android:toAlpha="1.0" />
    <scale android:duration="9000" android:fromXScale="0.0" android:toXScale="1.0"
        android:fromYScale="0.0" android:toYScale="1.0"
        android:interpolator="@android:anim/accelerate_decelerate_interpolator"
        android:pivotX="50%" android:pivotY="50%" android:fillAfter="false" />
    <translate android:duration="10000" android:fromXDelta="30" android:toXDelta="0"
        android:fromYDelta="30" android:toYDelta="0" />
    <rotate android:duration="10000" android:fromDegrees="0" android:toDegrees="+360"
        android:interpolator="@android:anim/accelerate_decelerate_interpolator"
        android:pivotX="50%" android:pivotY="50%" />
</set>

// TweenAnimActivity.java —— 加载并启动补间动画
private void start(int resId, final String desc) {
    Animation animation = AnimationUtils.loadAnimation(this, resId);  // ① 加载 XML 动画
    animation.setAnimationListener(new Animation.AnimationListener() {
        @Override public void onAnimationStart(Animation animation) {
            tvState.setText("开始播放：" + desc);
        }
        @Override public void onAnimationEnd(Animation animation) {
            tvState.setText("播放结束：" + desc + "（可再次点击重播）");
        }
        @Override public void onAnimationRepeat(Animation animation) { }
    });
    ivAndroid.clearAnimation();     // ② 清除上一次动画，避免叠加
    ivAndroid.startAnimation(animation);  // ③ 启动动画
}

// 按钮回调：根据点击的按钮加载对应动画资源
if (id == R.id.btnAlpha)          start(R.anim.alpha, "透明度动画 alpha：1.0 → 0.2");
else if (id == R.id.btnScale)     start(R.anim.scale, "缩放动画 scale：1.0 → 2.0 倍");
else if (id == R.id.btnTranslate) start(R.anim.translate, "平移动画 translate：(0,0) → (300,300)");
else if (id == R.id.btnRotate)    start(R.anim.rotate, "旋转动画 rotate：0° → 360°");
else if (id == R.id.btnCombo)     start(R.anim.anim_set, "组合动画 set：透明度+缩放+平移+旋转同时进行");
```

（2）粘贴动画开始、中间及结束阶段的运行截图，简述组合动画效果。




图 4-1 组合动画开始阶段（约 1.5s）
透明度、缩放刚开始，图像小而淡，并开始旋转




图 4-2 组合动画中间阶段（约 4.5s）
图像放大、半透明并已旋转约 180°




图 4-3 组合动画结束阶段（约 9.5s）

图像放大到原始大小、完全不透明，旋转接近 +360°

## 程序 5 SoundPool 音效播放

（1）写出 SoundPool 初始化、音频加载、按钮触发播放及资源释放的关键代码。

说明：5 个音效文件位于 res/raw（windbell.mp3、cuckoo.wav、doorbell.wav、phonebell.mp3、ring.wav），点击四个按钮播放对应音效，按任意按键在 onKeyDown 中播放按键音。

```
// SoundPoolActivity.java —— SoundPool 初始化、音频加载、按钮触发播放与资源释放
public class SoundPoolActivity extends Activity implements View.OnClickListener {
    private static final int MAX_STREAMS = 5;              // 最多同时播放 5 路
    private final int[] rawIds = {R.raw.windbell, R.raw.cuckoo, R.raw.doorbell, R.raw.phonebell};
    private final String[] soundNames = {"风铃声 windbell.mp3", "布谷鸟叫声 cuckoo.wav",
            "门铃声 doorbell.wav", "电话声 phonebell.mp3"};
    private final int[] soundIds = new int[4];
    private SoundPool soundPool;
    private int ringSoundId;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_soundpool);
        tvLog = (TextView) findViewById(R.id.tvSoundLog);

        // ① 初始化 SoundPool：设置最大同时播放数与音频属性
        AudioAttributes attributes = new AudioAttributes.Builder()
                .setUsage(AudioAttributes.USAGE_MEDIA)
                .setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION)
                .build();
```

```
            soundPool = new SoundPool.Builder()
                    .setMaxStreams(MAX_STREAMS)
                    .setAudioAttributes(attributes)
                    .build();

            // ② 加载音频：load() 返回音频 id（异步加载，完成后回调 onLoadComplete）
            soundPool.setOnLoadCompleteListener(new SoundPool.OnLoadCompleteListener() {
                @Override
                public void onLoadComplete(SoundPool soundPool, int sampleId, int status) {
                    log("音频加载完成 sampleId=" + sampleId + " status=" + status);
                }
            });
            for (int i = 0; i < rawIds.length; i++) {
                soundIds[i] = soundPool.load(this, rawIds[i], 1);   // priority = 1
                log("load " + soundNames[i] + " → soundId=" + soundIds[i]);
            }
            ringSoundId = soundPool.load(this, R.raw.ring, 1);      // 按键音

            int[] buttonIds = {R.id.btnSound1, R.id.btnSound2, R.id.btnSound3, R.id.btnSound4};
            for (int id : buttonIds) findViewById(id).setOnClickListener(this);
        }

        // ③ 按钮触发播放：play(左声道, 右声道, 优先级, 循环次数, 播放速率)
        @Override
        public void onClick(View v) {
            int index = -1;
            if (v.getId() == R.id.btnSound1) index = 0;
            else if (v.getId() == R.id.btnSound2) index = 1;
            else if (v.getId() == R.id.btnSound3) index = 2;
            else if (v.getId() == R.id.btnSound4) index = 3;
            if (index >= 0) {
                int streamId = soundPool.play(soundIds[index], 1, 1, 0, 0, 1);
                log("play " + soundNames[index] + " → streamID=" + streamId);
            }
        }

        // 按键音：按下任意按键时播放 ring.wav
        @Override
        public boolean onKeyDown(int keyCode, KeyEvent event) {
            if (soundPool != null) {
                int streamId = soundPool.play(ringSoundId, 1, 1, 0, 0, 1);
                log("onKeyDown(keyCode=" + keyCode + ") 播放按键音 ring.wav → streamID=" + streamId);
            }
            return super.onKeyDown(keyCode, event);
        }

        @Override
        protected void onDestroy() {
            super.onDestroy();
            if (soundPool != null) {
                soundPool.release();        // ④ 释放资源
                soundPool = null;
            }
        }
    }
```

（2）粘贴音效播放界面的运行截图，记录不同按钮对应的声音及实际播放结果。




图 5-1 程序 5 运行结果：4 个按钮 + 按键音（运行日志记录了加载与播放结果）

风铃声→windbell.mp3、布谷鸟叫声→cuckoo.wav、门铃声→doorbell.wav、电话声→phonebell.mp3、按键音→ring.wav（onKeyDown）

## 程序 6 MediaPlayer 音频播放

（1）写出 MediaPlayer 加载音频、播放、暂停、停止及资源释放的关键代码。

说明：音频文件 res/raw/xiaoge.mp3，MediaPlayer.create() 加载后由“播放/暂停/停止”三个按钮控制，界面实时显示播放状态与进度，onDestroy 中 release() 释放。

```
// AudioPlayerActivity.java —— MediaPlayer 加载音频、播放、暂停、停止与资源释放
public class AudioPlayerActivity extends Activity implements View.OnClickListener {
    private MediaPlayer mediaPlayer;
    private TextView tvState, tvProgress;
    private final Handler handler = new Handler(Looper.getMainLooper());

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_audio);
        tvState = (TextView) findViewById(R.id.tvAudioState);
        tvProgress = (TextView) findViewById(R.id.tvAudioProgress);

        // ① 加载音频：MediaPlayer.create() 根据 res/raw 资源创建播放器并完成 prepare
        mediaPlayer = MediaPlayer.create(this, R.raw.xiaoge);
        mediaPlayer.setOnCompletionListener(new MediaPlayer.OnCompletionListener() {
            @Override
            public void onCompletion(MediaPlayer mp) {
```

```
                tvState.setText("播放完成（onCompletion 回调），可点击“播放”重新播放");
            }
        });
        tvState.setText("已加载 res/raw/xiaoge.mp3，时长 " + format(mediaPlayer.getDuration()));

        findViewById(R.id.btnAudioPlay).setOnClickListener(this);
        findViewById(R.id.btnAudioPause).setOnClickListener(this);
        findViewById(R.id.btnAudioStop).setOnClickListener(this);

        handler.post(progressTask);      // 每 500ms 刷新一次播放进度
    }

    @Override
    public void onClick(View v) {
        if (v.getId() == R.id.btnAudioPlay) {
            if (!mediaPlayer.isPlaying()) {
                mediaPlayer.start();                // ② 播放
                tvState.setText("正在播放（isPlaying = true）");
            }
        } else if (v.getId() == R.id.btnAudioPause) {
            if (mediaPlayer.isPlaying()) {
                mediaPlayer.pause();                // ③ 暂停（再次 start 可继续）
                tvState.setText("已暂停（isPlaying = false），再次点击“播放”继续");
            }
        } else if (v.getId() == R.id.btnAudioStop) {
            mediaPlayer.stop();                     // ④ 停止
            try {
                mediaPlayer.prepare();              // 停止后必须 prepare() 才能再次播放
            } catch (IOException e) {
                e.printStackTrace();
            }
            mediaPlayer.seekTo(0);
            tvState.setText("已停止，并重新 prepare() 就绪");
        }
    }

    /** 每 500ms 刷新播放进度 */
    private void updateProgress() {
        if (mediaPlayer == null) return;
        int position = mediaPlayer.getCurrentPosition();
        int duration = mediaPlayer.getDuration();
        tvProgress.setText("播放进度：" + format(position) + " / " + format(duration));
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        handler.removeCallbacks(progressTask);
        if (mediaPlayer != null) {
            mediaPlayer.release();                  // ⑤ 释放资源
            mediaPlayer = null;
        }
    }
}
```

（2）粘贴播放、暂停和停止状态的运行截图，简述各操作的实际效果。




图 6-1　播放状态

正在播放，进度 00:01/00:57




图 6-2　暂停状态

已暂停，进度停在 00:02/00:57




图 6-3　停止状态

已停止并重新 prepare，进度归 0

# 程序 7 MediaPlayer 与 SurfaceView 视频播放

（1）写出视频源设置、SurfaceHolder 回调、MediaPlayer 与显示表面关联，以及播放、暂停和资源释放的关键代码。

说明：视频源为 res/raw/video.mp4（实验素材“实验 7”目录下的 video.mp4，时长 55s、分辨率 960×540）。SurfaceView 提供显示表面，SurfaceHolder.Callback 中把 MediaPlayer 与表面关联。

```
// VideoPlayerActivity.java —— MediaPlayer + SurfaceView 播放视频
public class VideoPlayerActivity extends Activity
        implements View.OnClickListener, SurfaceHolder.Callback {
    private SurfaceView surfaceView;
    private SurfaceHolder surfaceHolder;
    private MediaPlayer mediaPlayer;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_video);
        surfaceView = (SurfaceView) findViewById(R.id.svVideo);

        // ① SurfaceHolder 回调：surfaceCreated / surfaceChanged / surfaceDestroyed
        surfaceHolder = surfaceView.getHolder();
        surfaceHolder.addCallback(this);

        // ② 设置视频源：res/raw/video.mp4（也可用 setDataSource("/sdcard/xxx.mp4") 播放 SD 卡文件）
        mediaPlayer = MediaPlayer.create(this, R.raw.video);
        mediaPlayer.setLooping(false);
        mediaPlayer.setOnCompletionListener(new MediaPlayer.OnCompletionListener() {
            @Override
            public void onCompletion(MediaPlayer mp) {
                tvState.setText("播放完成（onCompletion 回调）");
            }
        });
        findViewById(R.id.btnVideoPlay).setOnClickListener(this);
        findViewById(R.id.btnVideoPause).setOnClickListener(this);
        findViewById(R.id.btnVideoStop).setOnClickListener(this);
    }

    // ③ 显示表面创建完成后，把 MediaPlayer 与 SurfaceView 关联
    @Override
    public void surfaceCreated(SurfaceHolder holder) {
        if (mediaPlayer != null) mediaPlayer.setDisplay(holder);
    }

    @Override
    public void surfaceChanged(SurfaceHolder holder, int format, int width, int height) {
        if (mediaPlayer != null) mediaPlayer.setDisplay(holder);
        tvProgress.setText("显示表面：" + width + " × " + height
                + "    视频：" + mediaPlayer.getVideoWidth() + " × " + mediaPlayer.getVideoHeight());
    }

    @Override
    public void surfaceDestroyed(SurfaceHolder holder) {
        if (mediaPlayer != null) mediaPlayer.setDisplay(null);   // ④ 取消关联，避免泄漏
    }

    // ⑤ 播放 / 暂停 / 停止
    @Override
    public void onClick(View v) {
        if (v.getId() == R.id.btnVideoPlay) {
            mediaPlayer.start();                // 播放
            tvState.setText("正在播放视频（isPlaying = true）");
```

```
        } else if (v.getId() == R.id.btnVideoPause) {
            if (mediaPlayer.isPlaying()) {
                mediaPlayer.pause();            // 暂停，再次点击“播放”继续
                tvState.setText("已暂停（isPlaying = false）");
            }
        } else if (v.getId() == R.id.btnVideoStop) {
            mediaPlayer.pause();
            mediaPlayer.seekTo(0);              // 停止：暂停并回到第 0 帧
            tvState.setText("已停止，画面回到第 0 帧");
        }
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        if (mediaPlayer != null) {
            mediaPlayer.release();              // ⑥ 释放资源
            mediaPlayer = null;
        }
    }
}
```

（2）粘贴视频播放与暂停状态的运行截图，注明视频文件的存放位置或来源。




图 7-1　视频播放状态（正在播放，位置 00:01/00:55，视频 960×540）




图 7-2　视频暂停状态（已暂停，画面停留在 00:02 处）

视频文件：res/raw/video.mp4（来源：实验素材“实验 7”目录）

## 二、思考题

## 2.1 习题

**1.如何设置画笔的颜色、笔触的宽度？如何绘制一个实心矩形？如何绘制字体大小为 25 的一行文本？如何绘制路径？**

1. 设置画笔颜色用 Paint.setColor(int color)，常用 Color.RED、Color.rgb(r,g,b)、Color.argb(a,r,g,b)，也可用 setShader 设置渐变；设置笔触宽度用 Paint.setStrokeWidth(float width)，单位为像素，仅当 setStyle(Paint.Style.STROKE) 描边时线宽才可见。

2. 绘制实心矩形：paint.setStyle(Paint.Style.FILL); canvas.drawRect(left, top, right, bottom, paint) 或 canvas.drawRect(RectF, paint)；若先 setStyle(Paint.Style.STROKE) 再 drawRect 则是空心矩形。

3. 绘制字体大小为 25 的文本：paint.setTextSize(25); canvas.drawText("文本", x, y, paint)，其中 (x, y) 是文字基线的起点；需要居中时可用 paint.measureText() 或 Paint.Align.CENTER。

4. 绘制路径：Path path = new Path(); path.moveTo(x0,y0); path.lineTo(x1,y1); path.lineTo(...); path.close(); canvas.drawPath(path, paint)；Path 还支持 addRect/addCircle/quadTo/cubicTo 等，配合 canvas.drawTextOnPath() 还可沿路径绘制文本。

**2. 请描述 Matrix 类的旋转变换方法 setRotate(60)、postRotate(60)、preRotate(60)以及 setRotate(60,100,150)的区别是什么？以及如何对一幅图像进行平移变换或缩放变换或扭曲变换并显示之？**

1. 三个方法都绕原点 (0,0) 旋转 60°，区别在于与已有变换的叠加方式：

（1）setRotate(60)：先把矩阵重置为单位矩阵，再设置为旋转 60°，会清除之前的平移/缩放到矩阵中的设置；

（2）postRotate(60)：后乘旋转矩阵，M' = R × M，相当于先执行矩阵已有变换，再整体绕原点旋转；常与 setTranslate 等配合使用；

（3）preRotate(60)：前乘旋转矩阵，M' = M × R，相当于先在原坐标系中旋转，再执行矩阵已有变换；

（4）setRotate(60, 100, 150)：重置矩阵后绕指定点 (100,150)（而不是原点）旋转 60°。

2. 平移：Matrix m = new Matrix(); m.setTranslate(dx, dy); canvas.drawBitmap(bitmap, m, paint)（等价于 drawBitmap(bitmap, dx, dy, paint)）。

3. 缩放：m.setScale(sx, sy); m.postTranslate(dx, dy); canvas.drawBitmap(bitmap, m, paint)；注意第二次要用 postTranslate，若再用 setTranslate 会把缩放覆盖掉。

4. 扭曲：m.setSkew(kx, ky); m.postTranslate(dx, dy); canvas.drawBitmap(bitmap, m, paint)，也可直接 m.setSkew(kx, ky, px, py) 指定中心点。

**3. 如何制作一个帧动画？**

制作帧动画的步骤如下：

（1）准备一组连续动作的图片（如 p01.png~p10.png），放入 res/drawable；

（2）在 res/anim 目录下新建 XML（如 album_list.xml），根元素为 <animation-list>，用 android:oneshot="false" 设置为循环播放，每个 <item android:drawable="@drawable/p01" android:duration="1000"/> 指定一帧图片和显示时长；

（3）在布局中用 ImageView 显示，可设置 android:background="@anim/album_list"，也可在代码中 iv.setBackgroundResource(R.anim.album_list)）；

（4）代码中通过 AnimationDrawable ad = (AnimationDrawable) iv.getBackground(); 取得动画对象调用 ad.start() 开始播放、ad.stop() 停止播放；

（5）注意 start() 必须在窗口可见之后调用（如按钮单击回调或 onWindowFocusChanged 中），否则动画不会播放。

**4.如何制作一个补间动画？**

制作补间动画的步骤如下：

（1）在 res/anim 目录下新建动画 XML：<alpha>（透明度 fromAlpha/toAlpha）、<scale>（缩放 fromXScale/toScale、pivotX/pivotY 旋转轴心）、<translate>（平移 fromXDelta/toXDelta）、<rotate>（旋转 fromDegrees/toDegrees），公共属性有 duration（时长）、interpolator（速度插值器）fillAfter（结束后是否保持）；

（2）需要同时播放多个动画时，用一个 <set> 作为根元素把各个动画包起来，并可用 shareInterpolator 指定是否共用插值器；

（3）在代码中用 Animation animation = AnimationUtils.loadAnimation(this, R.anim.tween_anim) 加载动画资源；

（4）调用 view.startAnimation(animation) 启动动画；重复播放前先 clearAnimation() 清除上一次动画；

（5）补间动画只是改变 View 的绘制效果，并不改变 View 的真实属性（left/top/size），若要真正改变属性应使用属性动画（ObjectAnimator/ValueAnimator）。

**5.使用 SoundPool 播放音频的主要步骤是什么？**

使用 SoundPool 播放音频的主要步骤：

（1）创建 SoundPool 对象：SoundPool soundPool = new SoundPool.Builder().setMaxStreams(5).setAudioAttributes(audioAttributes).build()；旧版本可用 new SoundPool(5, AudioManager.STREAM_MUSIC, 0)；

（2）加载音频：int soundId = soundPool.load(this, R.raw.doorbell, 1)，load() 是异步的，返回音频

id，可注册 setOnLoadCompleteListener 在 status==0 时得知加载完成；

（3）播放音频：soundPool.play(soundId, leftVolume, rightVolume, priority, loop, rate)，返回 streamId，同一音效可同时播放多路；

（4）控制与释放：可用 soundPool.stop(streamId)、soundPool.pause(streamId) 控制，不再使用时调用 soundPool.release() 释放资源；

（5）SoundPool 适合短小的音效（如按键音、门铃），不适合长时间音乐，长音频应使用 MediaPlayer。

**6. 如何使用 MediaPlayer 播放音频、暂停播放、停止播放音频？**

使用 MediaPlayer 播放音频的步骤：

（1）创建并加载：MediaPlayer mp = MediaPlayer.create(this, R.raw.xiaoge)（内部已完成 prepare），也可 new MediaPlayer() 后 setDataSource() + prepare()；

（2）播放：mp.start()，用 mp.isPlaying() 判断是否正在播放；

（3）暂停：mp.pause()，再次调用 mp.start() 可以从暂停位置继续播放；

（4）停止：mp.stop()，停止后播放器处于 Stopped 状态，必须重新 prepare() 才能再次 start()；

（5）其他：setOnCompletionListener 监听播放完成，seekTo() 定位，getCurrentPosition()/getDuration() 获取进度与时长；

（6）释放：在 Activity 的 onDestroy() 中调用 mp.release() 释放底层解码器资源，避免内存泄漏。

**7. 如何使用 VideoView 播放视频、暂停播放、停止播放？**

使用 VideoView 播放视频（控件已封装好 MediaPlayer 与 SurfaceView）：

（1）布局中添加 <VideoView android:id="@+id/vv" .../>；

（2）设置视频源：vv.setVideoPath("/sdcard/beargolf.mp4")；也可 vv.setVideoURI(Uri.parse("android.resource://" + getPackageName() + "/" + R.raw.video)) 播放 res/raw 中的视频，或传入网络 URL；

（3）可选 vv.setMediaController(new MediaController(this)) 显示进度条控制条；

（4）播放：vv.start()；暂停：vv.pause()；继续：vv.resume()；

（5）停止：vv.stopPlayback()（停止并释放播放器），若要重新播放需再次 setVideoPath 再 start()；

（6）在 onDestroy() 中调用 vv.stopPlayback() 释放资源。

**8. 如何使用 MediaPlayer+SurfaceView 播放视频、暂停播放、停止播放？**

使用 MediaPlayer + SurfaceView 播放视频的步骤：

（1）布局中添加 SurfaceView（MediaPlayer 的画面会输出到它的 Surface 上）；

（2）在 Activity 中实现 SurfaceHolder.Callback，通过 surfaceView.getHolder().addCallback(this) 注册回调；

（3）创建 MediaPlayer：new MediaPlayer() 或 MediaPlayer.create(this, R.raw.video)，setDataSource(path) 指定视频（SD 卡路径如 "/sdcard/beargolf.mp4"）；

（4）在 surfaceCreated(SurfaceHolder holder) 中调用 mediaPlayer.setDisplay(holder)，把 MediaPlayer 与显示表面关联（必须在 Surface 创建之后）；

（5）准备并播放：mediaPlayer.prepare()（或 prepareAsync）+ mediaPlayer.start()；

（6）暂停：mediaPlayer.pause()，继续：mediaPlayer.start()；停止：mediaPlayer.pause() + seekTo(0)；

（7）在 surfaceDestroyed() 中 setDisplay(null)，在 onDestroy() 中 stop()/release() 释放资源。相比 VideoView，MediaPlayer + SurfaceView 更灵活，可自行控制渲染与布局。

## 2.2 实验小结

**1 编程遇到的问题及解决方案（编程中遇到的问题（主要指对属性的理解有误，以及程序调试过程等，分条写）**

1. 自定义 View 添加方式错误导致不显示：最初只在布局中写了普通 View，忘记了自定义 View 需要在布局里使用全类名（<com.example.graphicsmedia.ShapeView .../>），或者用 FrameLayout.addView(new MyView(this)) 动态添加，两种方式必须与 setContentView 的布局一致。

2. 画笔设置了宽度却画不出线：Paint.Style.FILL 时 setStrokeWidth 不生效，只有 STROKE 样式才有线宽；同时 setStyle 与 setStrokeWidth 的顺序没有影响，但 setColor 必须在 draw 之前设置。

3. Matrix 变换结果与预期不符：一开始每次都调用 setRotate/setScale，会清空前面的变换（setXXX 会重置矩阵），改为先 setScale/setRotate，再用 postTranslate 叠加平移后才得到正确结果

4. 帧动画点击按钮不播放：AnimationDrawable.start() 在 onCreate() 里调用无效，因为此时窗口还不可见；改为在按钮 onClick 中启动后正常。另外忘记给 ImageView 设置背景动画资源也会黑屏

5. 补间动画结束后图像跳回原位：补间动画只改变绘制效果，结束时会恢复；需要保留最终状态可设置 fillAfter="true"，或者改用属性动画。

6. SoundPool 第一次点击没有声音：load() 是异步加载，未加载完成就 play() 会返回 0；通过 setOnLoadCompleteListener 确认加载完成（status==0）后再播放即可。

7. MediaPlayer 停止后再次播放抛 IllegalStateException：调用 stop() 后播放器处于 Stopped 状态，必须重新 prepare() 才能 start()；改成 stop() + prepare() + seekTo(0) 后解决。

8. 视频播放黑屏：在 surfaceCreated() 之前调用 setDisplay() 无效，必须在 SurfaceHolder.Callback 的 surfaceCreated 回调中再 setDisplay(holder)，并且 MediaPlayer 要先准备完成才能输出画面。

...

...

**2 实验收获及体会（本次实验掌握了哪些知识点，学到了什么？可以进行总结在这里，分条写，不要堆在一起）**

1. 掌握了 Paint 的常用属性：颜色（setColor/setARGB）、样式（FILL/STROKE）、笔触宽度、抗锯齿、线帽、文字大小与加粗，以及 Canvas 的 drawRect/drawCircle/drawOval/drawArc/drawLine/drawPath/drawText 等绘图 API，理解了 Canvas 的坐标系与 save()/restore() 的作用。

2. 掌握了 Matrix 的变换原理：平移 setTranslate、旋转 setRotate、缩放 setScale、扭曲 setSkew，以及 pre/post 前缀表示与已有变换的叠加顺序，会用 Matrix + drawBitmap 显示变换后的图像。

3. 掌握了帧动画的制作流程：animation-list 资源 + AnimationDrawable 的 start()/stop()，理解了 oneshot、duration 的含义以及必须在窗口可见后启动的限制。

4. 掌握了补间动画的制作：alpha/scale/translate/rotate 四类 XML 资源及 set 组合动画，AnimationUtils.loadAnimation + startAnimation 的使用，以及 pivot、interpolator、fillAfter 等属性的作用。

5. 掌握了 SoundPool 播放短音效的完整流程（Builder 初始化、异步 load、play、release），以及 MediaPlayer 播放音频的 create/start/pause/stop/prepare/release 状态机。

6. 掌握了 MediaPlayer + SurfaceView 播放视频的方法：SurfaceHolder.Callback 生命周期、setDisplay() 关联、播放/暂停/停止控制与资源释放，体会到 SurfaceView 与 VideoView 的区别。

7. 体会到多媒体开发中“资源必须及时释放”的重要性（MediaPlayer.release、SoundPool.release、handler.removeCallbacks），否则容易造成内存泄漏和后台耗电。

**3 不足之处及下一步需改进的地方（实验不足之处，以及需要改进的地方，分条写，不要堆在一起）**

1. 绘图程序中图形坐标是按 1080×2400 屏幕写死的像素值，虽然做了等比缩放，但换到其他分辨率或横屏时仍可能出现文字重叠；下一步应结合 dp、屏幕密度和 ConstraintLayout 做自适应布局。

2. 帧动画把 10 张图片全部解码到内存中，图片较大时容易占用过多内存；后续可以改用更小的 WebP 图片、按需加载，或者改用视频/属性动画实现相同效果。

3. 音效与音频界面功能较单一，没有音量、进度条、循环模式等控制；可以增加 SeekBar 拖动定位、音量调节、播放列表，并处理 AudioFocus 与耳机拔出等音频焦点事件。

4. 程序对异常情况的处理不足（如视频文件不存在、格式不支持、SD 卡无权限等），后续应补充 try-catch、状态判断与用户提示；同时可以把七个程序改为一个带 Fragment/导航的完整应用，界面更统一。